INSTITUTO FEDERAL DA PARAÍBA – IFPB

DISCIPLINA: SISTEMAS OPERACIONAIS

STREAMING SOB CONTENÇÃO DE RECURSOS

Avaliação de desempenho em ambiente Linux

Omar Henrique e Davi Patrício

João Pessoa – PB
2026

Abstract

Servidores de streaming de vídeo são sensíveis à disputa por CPU, pois atrasos na entrega de segmentos elevam a latência de cauda e podem causar *rebuffering*. Este trabalho avalia, em Linux nativo, como a contenção de CPU gerada pelo *stress-ng* afeta um servidor Nginx e compara cinco modos de mitigação: nenhum isolamento, afinidade de CPU, cgroups v2 (pesos), prioridade (*nice*) e uma mitigação adaptativa proposta, na qual um controlador mede a espera do Nginx na fila de execução do escalonador (via `schedstat` e eBPF) e ajusta limites de CPU em laço fechado. O planejamento prevê 5 modos, 4 níveis de contenção proporcionais ao número de CPUs lógicas e 10 repetições por configuração, com mediana, intervalos de confiança por *bootstrap* e teste de Mann–Whitney com correção de Holm. Os resultados apresentados aqui são do **estudo piloto** (3 repetições, 12 CPUs lógicas): a espera do Nginx na fila cresce cerca de 200 vezes (de 0,6 para 123 ms/s) entre o *baseline* e a contenção máxima, e todos os mecanismos a reduzem (de 5 a 15 vezes); porém, o P99 não acompanhou essa espera (correlação de Spearman $\rho = -0{,}03$), e a mitigação adaptativa protegeu o servidor ao custo de reduzir a CPU da carga concorrente de cerca de 10 para 2,7 CPUs, ao contrário do isolamento por pesos. Com 3 repetições nenhum teste pode ser significativo; a coleta definitiva fará essa verificação.

Contents

1 Introdução **3**
1.1 Questões de pesquisa . . . 3
1.2 Hipóteses . . . 3
1.3 Contribuições . . . 3

2 Fundamentação Teórica **4**
2.1 Streaming de vídeo . . . 4
2.2 Contenção de CPU e fila de execução . . . 4
2.3 Isolamento e priorização no Linux . . . 4
2.4 eBPF e observabilidade do escalonador . . . 4
2.5 Mitigação adaptativa . . . 5

3 Trabalhos Relacionados **5**

4 Metodologia Experimental **6**
4.1 Arquitetura do ambiente . . . 6
4.2 Hardware e software . . . 7
4.3 Fatores e configurações . . . 7
4.4 Variáveis . . . 8
4.5 Carga, métricas e procedimento . . . 8
4.6 Plano estatístico . . . 8

5 Resultados **8**
5.1 Contenção e espera na fila (RQ1 e RQ3) . . . 9
5.2 Mecanismos estáticos (RQ2) . . . 10
5.3 Mitigação adaptativa e custo para a carga (RQ4) . . . 10

6 Discussão **12**

7 Ameaças à Validação **13**

8 Conclusões **13**

1 Introdução

O streaming de vídeo depende da capacidade do servidor de atender requisições de forma contínua. Quando a CPU é disputada por outros processos, o servidor passa mais tempo esperando para executar, o que aumenta a latência de cada resposta e, principalmente, a latência de cauda (P95/P99), que é a que interrompe a reprodução.

Este projeto analisa o impacto da contenção de CPU sobre um servidor de streaming, compara mecanismos de isolamento e priorização do Linux e propõe uma **mitigação adaptativa**: um controlador que detecta a contenção a partir de um sinal do escalonador, coletado com eBPF, e ajusta automaticamente os limites de CPU. O experimento é realizado em máquina física com Linux nativo, sem máquina virtual. O Nginx serve segmentos de vídeo, o *stress-ng* gera a contenção e o projeto segue três etapas: *baseline*, perturbação e mitigação.

1.1 Questões de pesquisa

RQ1 Como diferentes níveis de contenção de CPU afetam a latência (P95/P99) e o rebuffering de um servidor de streaming?

RQ2 Os mecanismos estáticos de isolamento (afinidade, cgroups e prioridade) reduzem esse impacto?

RQ3 A espera do Nginx na fila de execução, medida com eBPF, explica o aumento da latência?

RQ4 Uma mitigação adaptativa reduz a latência com menor custo para a carga concorrente do que o isolamento estático?

1.2 Hipóteses

H1 O P99 e o rebuffering aumentam com o nível de contenção.

H2 Ao menos um mecanismo estático reduz o P99 em relação ao cenário sem isolamento sob contenção alta.

H3 A espera na fila de execução cresce com a contenção e acompanha o P99.

H4 A mitigação adaptativa reduz o P99 em relação ao cenário sem isolamento, penalizando menos a carga concorrente do que um limite fixo.

1.3 Contribuições

(i) Um ambiente reprodutível e automatizado (orquestrador, cliente sintético, análise); (ii) a medição da espera do Nginx na fila de execução por duas vias independentes

(`schedstat` e eBPF); (iii) um controlador adaptativo de código aberto; e (iv) uma avaliação que expõe quando a espera na fila é, ou não, um bom indicador de latência de cauda.

2 Fundamentação Teórica

2.1 Streaming de vídeo

O conteúdo é dividido em segmentos solicitados por HTTP. Atrasos na entrega comprometem a continuidade da reprodução. O cliente usado aqui é sintético: não reproduz vídeo real (HLS/DASH); requisita segmentos de 400 KB e usa como proxy de *rebuffering* a fração de segmentos entregues acima de um limite de tempo.

2.2 Contenção de CPU e fila de execução

Cada CPU lógica mantém uma fila de execução (*run queue*). Quando há mais tarefas prontas que CPUs, uma tarefa espera antes de executar e pode ser preemptada. A cadeia de efeitos esperada é: contenção → espera na fila → aumento da latência por requisição → aumento do P99 → rebuffering. O escalonador padrão do kernel utilizado (7.0) é baseado no EEVDF [6], que prioriza tarefas com menor tempo de espera acumulado, o que favorece tarefas curtas e que dormem com frequência, como um servidor ocioso entre requisições.

2.3 Isolamento e priorização no Linux

A afinidade de CPU (`taskset`, `sched_setaffinity`) restringe em quais CPUs um processo executa. Os cgroups v2 [7] controlam o uso proporcional (`cpu.weight`, válido quando há disputa) e limitam o uso absoluto (`cpu.max`). A prioridade (*nice*) altera o peso do processo no escalonador. Nenhum desses mecanismos é adaptativo: seus parâmetros são fixos durante a execução.

2.4 eBPF e observabilidade do escalonador

O eBPF executa programas verificados no kernel, associados a eventos como os do escalonador, sem alterar o kernel [5]. A espera de uma tarefa na fila pode ser medida pelo intervalo entre o *wakeup* e o momento em que ela ganha a CPU, ou lida do contador acumulado `run_delay` em `/proc/PID/schedstat`. Um programa de *tracing* apenas observa: a ação é feita por um controlador em espaço de usuário. O *sched_ext* [9] permite escrever o próprio escalonador em eBPF; está disponível no kernel utilizado, mas

não é avaliado neste trabalho. Complementarmente, o PSI [8] informa a pressão de CPU do sistema.

2.5 Mitigação adaptativa

É um laço fechado: medir a espera do Nginx → detectar contenção → decidir por limiar → ajustar `cpu.weight` e `cpu.max` do grupo que gera a carga → medir de novo. O controlador amostra a espera a cada 200 ms, sobe um degrau quando o sinal excede o limiar alto por 2 janelas seguidas e relaxa um degrau quando fica abaixo do limiar baixo por 10 janelas (histerese). Os degraus têm duas fases: no degrau 1 só os pesos mudam (Nginx 400 e carga 10, equivalente ao isolamento estático por pesos); nos degraus 2 a 4 acrescenta-se um teto de CPU para a carga (50%, 25% e 10% das CPUs). Assim, a ação mais barata para a carga vem primeiro. No piloto descrito na Seção 5, porém, o primeiro degrau já incluía o teto de 50%; a versão em duas fases foi implementada depois dele. Atua apenas sobre os grupos do experimento e restaura os valores ao final.

3 Trabalhos Relacionados

A importância da latência de cauda em serviços interativos é discutida por Dean e Barroso [1], que mostram como pequenas variações em componentes se amplificam em sistemas distribuídos. Lozi *et al.* [2] mostram que o escalonador do Linux pode deixar CPUs ociosas enquanto há tarefas esperando, o que torna a espera na fila um sinal relevante de desempenho.

Sobre contenção entre cargas colocalizadas, Liu *et al.* [3] observam que contêineres colocalizados podem sofrer lentidão de até 5× por causa da diferença entre a CPU reservada e a CPU efetivamente obtida, que atribuem ao caráter conservador de trabalho do escalonador CFS, e propõem o rKube para corrigi-la. Jeffery *et al.* [4] atribuem latências de cauda elevadas ao excesso de *threads* de aplicações que ignoram sua cota de CPU e propõem um *threadpool* que ajusta seus trabalhadores à cota disponível. Ferramentas de observabilidade com eBPF para medir a latência de fila do escalonador estão descritas por Gregg [5].

Este trabalho difere dos anteriores em três pontos: compara, sob a mesma carga e na mesma máquina, mecanismos estáticos do Linux com uma mitigação adaptativa; usa a espera do próprio servidor na fila, medida por duas vias, como sinal de controle; e relata o custo da mitigação para a carga concorrente, além do ganho de latência.

4 Metodologia Experimental

4.1 Arquitetura do ambiente

O experimento roda em máquina física com Linux nativo. O orquestrador configura cada cenário, inicia o *stress-ng*, o monitor eBPF e o controlador, executa o cliente e coleta os resultados (Figura 1).

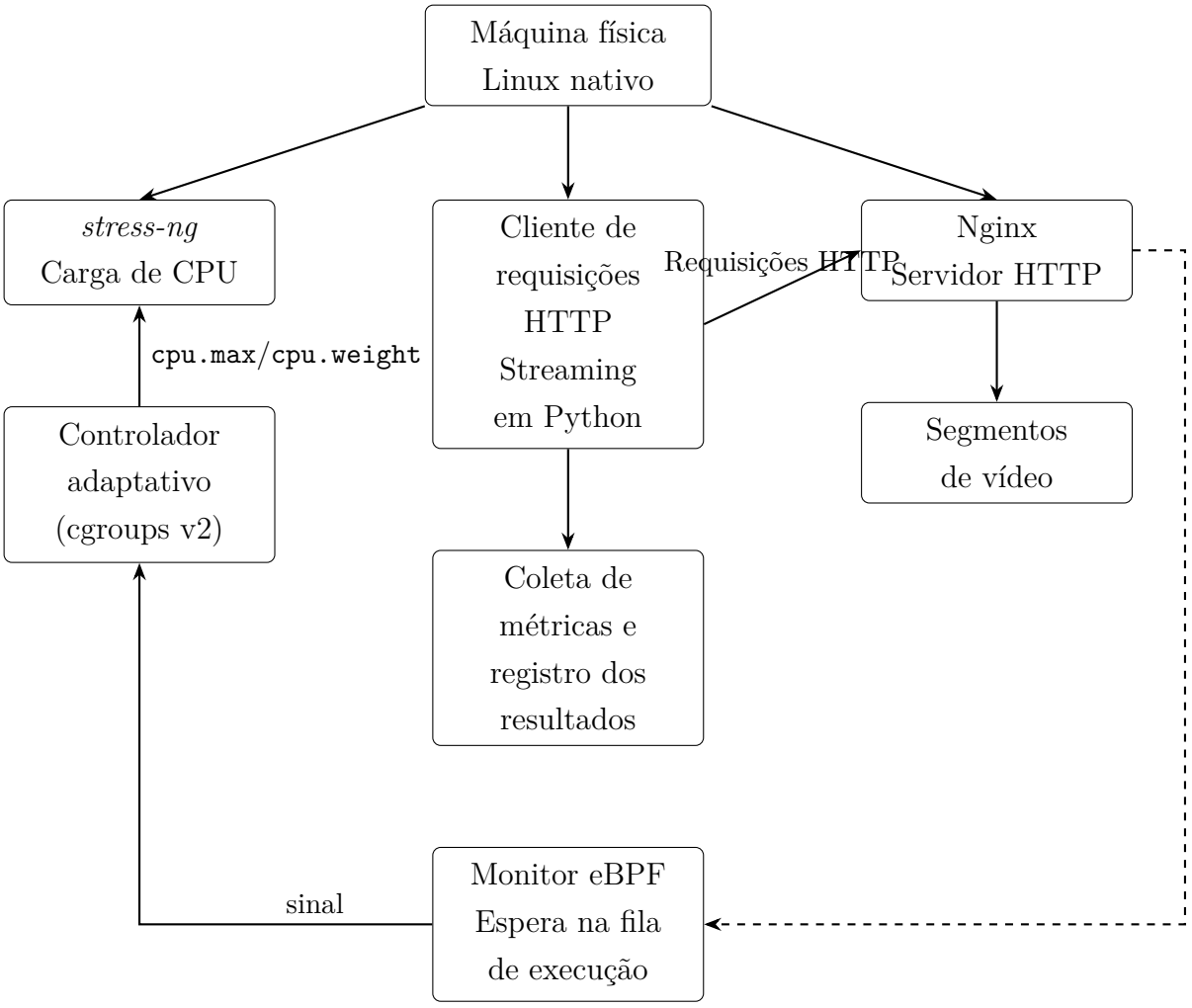


Figure 1: Arquitetura do ambiente experimental em Linux nativo, com o monitor eBPF e o controlador adaptativo (setas tracejadas: observação).

4.2 Hardware e software

Table 1: Ambiente experimental.

Componente	**Especificação**
Processador	AMD Ryzen 5 5500U, 6 núcleos / 12 threads
Processadores lógicos (N)	12
Memória RAM	8 GB
Kernel	7.0.0-43-generic (Ubuntu, distribuição *resolute*)
Suporte a *sched_ext*	Sim
Ambiente	Máquina física, Linux nativo

Softwares: Nginx (1 *worker*, `sendfile` desligado), Python (cliente, orquestrador e controlador), *stress-ng*, *bpftrace*, *taskset*, cgroups, *nice* e `/proc` (PSI, `schedstat`, trocas de contexto).

4.3 Fatores e configurações

Cinco modos: sem isolamento; afinidade (Nginx na CPU 0 e a carga nas demais); cgroups v2 por pesos (4096 para o servidor e 128 para a carga, via `cpu.shares`/`cpu.weight`); prioridade (*nice* -15 para o Nginx); e adaptativo. Quatro níveis de contenção proporcionais ao número K de CPUs disponíveis ao experimento: 0, $K/2$, K e $2K$ processos do *stress-ng*. São $5 \times 4 = 20$ configurações. No piloto, $K = N = 12$ (0, 6, 12 e 24 processos). Como a contenção quase não afetou o P99 com 12 CPUs livres, a coleta definitiva **confina** Nginx, cliente e *stress-ng* às CPUs 0 a 3 ($K = 4$; níveis 0, 2, 4 e 8), mantendo o orquestrador e o monitor nas demais, e fixa o governador de frequência em *performance* (restaurado ao final).

4.4 Variáveis

Table 2: Variáveis do experimento.

Tipo	Variáveis
Independentes	modo (5 níveis); nível de contenção (0, $N/2$, N, $2N$)
Dependentes	P95 e P99 da latência; *rebuffering*; vazão; espera do Nginx na fila (ms por segundo); CPU obtida pela carga (CPUs); trocas de contexto; PSI; tempo de reação do controlador
Controladas	conteúdo (20 segmentos de 400 KB); 20 espectadores a cerca de 5 req/s cada; duração; máquina e kernel; governador de CPU; CPUs do experimento (K); notebook na tomada

4.5 Carga, métricas e procedimento

O cliente mantém conexões persistentes e registra a latência de cada requisição; o P95 e o P99 são calculados por execução. A espera na fila é o incremento de `run_delay` do *worker* do Nginx dividido pela duração (ms por segundo). O custo da carga é o tempo de CPU consumido pelo grupo do *stress-ng* dividido pela duração (em número de CPUs). A ordem das execuções é sorteada com semente registrada; há aquecimento de 2 s antes da medição e esfriamento entre execuções, e nenhuma rodada sobrescreve outra. O monitor eBPF (`fila_cpu.bt`) foi validado em uma execução e concordou com o `schedstat` (19,2 contra 18,8 ms/s).

4.6 Plano estatístico

A unidade de análise é a execução (n = repetições por configuração). Para cada métrica reportam-se a mediana, o intervalo interquartil e o IC de 95% da mediana por *bootstrap* percentil [10] (10 000 reamostragens). Cada modo é comparado ao *baseline* sem isolamento no mesmo nível pelo teste de Mann–Whitney [11] bilateral, com tamanho de efeito pelo δ de Cliff [12] e correção de Holm [13] por família (métrica $\times$ nível). A relação entre espera e P99 é avaliada pela correlação de Spearman. Com $n = 3$ vs. 3, o menor p possível do teste é 0,1; por isso o piloto não permite afirmar significância, e a coleta definitiva usa 10 repetições de 30 s (200 execuções).

5 Resultados

Nota: os resultados abaixo são do **estudo piloto** (rodada 5: 60 execuções, 3 repetições de 8 s por configuração, 20 espectadores, 12 CPUs lógicas). Eles devem ser substituídos pelos da coleta definitiva (10 repetições de 30 s) reexecutando `analise/estatistica.py`.

5.1 Contenção e espera na fila (RQ1 e RQ3)

Sem isolamento, a espera mediana do Nginx na fila foi de 0,6 ms/s sem estresse, 4,4 (6 processos), 41 (12) e 123 ms/s (24), crescimento de cerca de 200 vezes (Tabela 3, Figura 2). A vazão caiu de 98,3 para 93,2 req/s.

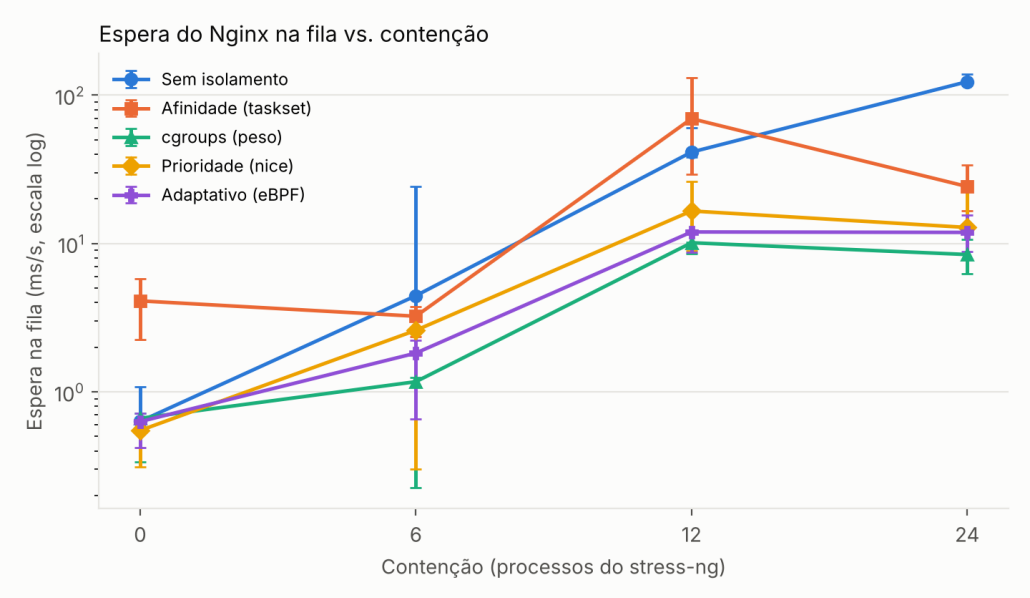


Figure 2: Espera do Nginx na fila de execução por nível de contenção (mediana e IC 95%, escala logarítmica).

Table 3: Espera do Nginx na fila de execução (ms/s): mediana [IC 95%].

Modo	**0 proc.**	**6 proc.**	**12 proc.**	**24 proc.**
Sem isolamento	0.6 [0.5; 1.1]	4.4 [1.8; 24.2]	41.3 [38.3; 60.1]	123.3 [123.3; 138.4]
Afinidade (taskset)	4.1 [2.2; 5.8]	3.2 [2.4; 3.7]	69.5 [29.2; 131.1]	24.2 [16.5; 33.7]
cgroups (peso)	0.7 [0.3; 0.7]	1.2 [0.2; 1.2]	10.1 [8.5; 10.1]	8.4 [6.3; 10.7]
Prioridade (nice)	0.5 [0.3; 0.6]	2.6 [0.3; 2.6]	16.6 [8.9; 26.2]	12.8 [11.8; 22.7]
Adaptativo (eBPF)	0.6 [0.4; 0.7]	1.8 [0.7; 2.2]	12.0 [8.7; 12.3]	11.9 [8.8; 15.6]

O P99, entretanto, não acompanhou essa espera (Tabela 4, Figura 3): sem isolamento foi 16,4 ms sem estresse, 10,7, 5,7 e 13,7 ms nos níveis 6, 12 e 24. A correlação de Spearman entre espera e P99 nas 60 execuções foi $\rho = -0{,}03$ (IC 95% $[-0{,}32; 0{,}25]$, $p = 0{,}81$). O *rebuffering* (segmentos acima de 20 ms) foi praticamente nulo em todas as configurações e, por isso, não discrimina os modos.

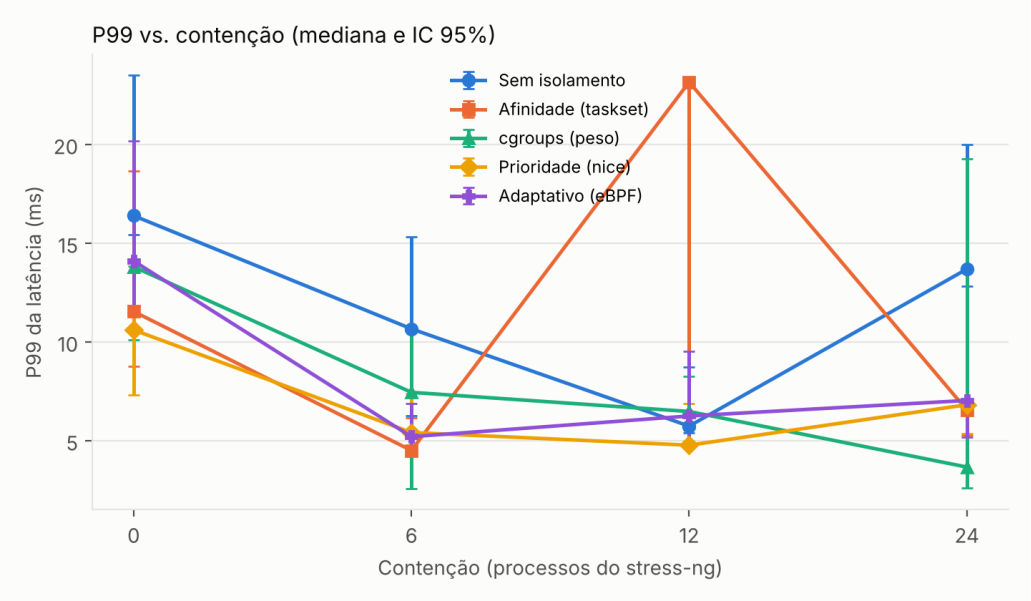


Figure 3: P99 da latência por nível de contenção e modo (mediana e IC 95% do bootstrap, $n = 3$).

Table 4: P99 da latência (ms): mediana [IC 95% bootstrap].

Modo	**0 proc.**	**6 proc.**	**12 proc.**	**24 proc.**
Sem isolamento	16.4 [15.4; 23.5]	10.7 [6.3; 15.3]	5.7 [5.4; 8.7]	13.7 [12.8; 20.0]
Afinidade (taskset)	11.5 [8.8; 18.7]	4.5 [4.4; 6.2]	23.2 [5.8; 23.2]	6.5 [5.3; 7.0]
cgroups (peso)	13.8 [10.1; 13.8]	7.4 [2.6; 10.8]	6.5 [4.8; 8.2]	3.7 [2.6; 19.3]
Prioridade (nice)	10.6 [7.3; 14.2]	5.4 [4.3; 7.4]	4.8 [4.8; 6.9]	6.8 [5.4; 7.0]
Adaptativo (eBPF)	14.1 [11.8; 20.2]	5.2 [4.7; 6.9]	6.3 [5.0; 9.5]	7.0 [5.2; 7.1]

5.2 Mecanismos estáticos (RQ2)

Com 24 processos, todos os mecanismos reduziram a espera de 123 para 8–24 ms/s (cgroups 8,4; adaptativo 11,9; prioridade 12,8; afinidade 24,2). O P99 mediano foi menor que o do *baseline* (13,7 ms) em todos: cgroups 3,7; afinidade 6,6; prioridade 6,8; adaptativo 7,0 ms. Com $n = 3$, nenhuma dessas diferenças é significativa após a correção de Holm ($p_{\text{Holm}} = 0{,}4$, $|\delta|$ de Cliff entre 0,56 e 1,0). O *baseline* sem estresse teve P99 acima do de 6 processos em quase todos os modos, comportamento já observado nos pilotos anteriores.

5.3 Mitigação adaptativa e custo para a carga (RQ4)

O controlador não agiu com 0 e 6 processos. Com 12 e 24, atuou em média 2 a 3 vezes e reagiu em 0,4 a 0,6 s após o início do tráfego (mediana 0,61 s; Figura 4). A espera caiu de 123 para 12 ms/s no nível 24. O custo foi alto para a carga: ela obteve 2,7 CPUs, contra 10,2 sem isolamento e 10,8 com cgroups por pesos (Tabela 5, Figura 5). Ou seja, o

isolamento por pesos protegeu o Nginx tão bem quanto o adaptativo, sem reduzir a CPU da carga.

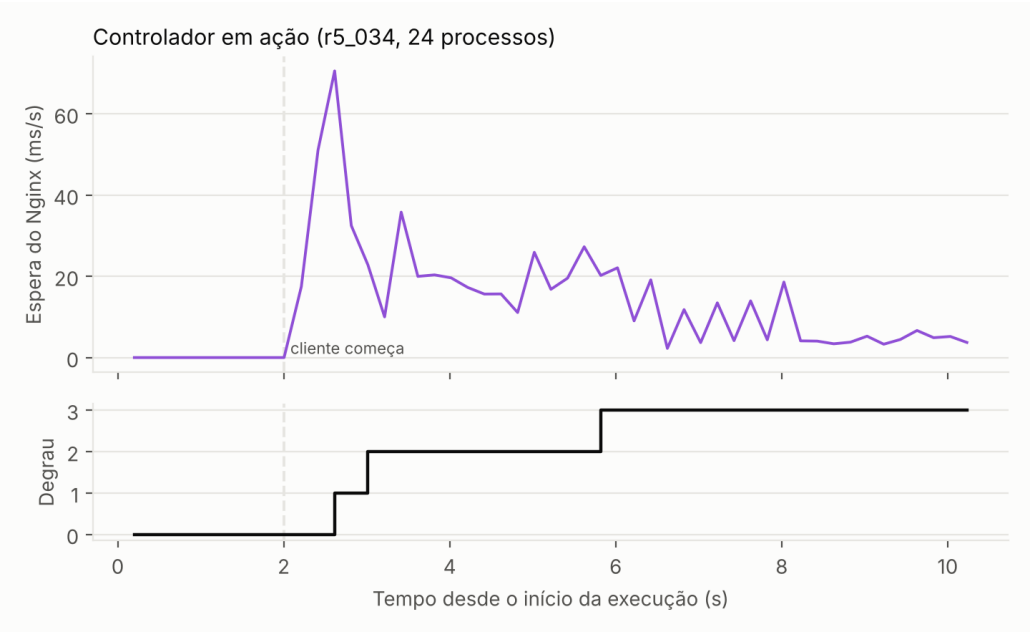


Figure 4: Execução com 24 processos: espera do Nginx e degrau do controlador. O tráfego começa em 2 s, após o aquecimento.

Table 5: CPU obtida pela carga do stress-ng (CPUs): mediana [IC 95%].

Modo	**0 proc.**	**6 proc.**	**12 proc.**	**24 proc.**
Sem isolamento	0.0 [0.0; 0.0]	6.0 [5.9; 6.0]	10.2 [10.2; 10.8]	10.2 [10.0; 10.3]
Afinidade (taskset)	0.0 [0.0; 0.0]	6.0 [6.0; 6.0]	10.1 [9.5; 10.4]	10.0 [9.7; 10.3]
cgroups (peso)	0.0 [0.0; 0.0]	6.0 [6.0; 6.0]	10.2 [9.9; 10.3]	10.8 [10.0; 10.9]
Prioridade (nice)	0.0 [0.0; 0.0]	6.0 [6.0; 6.0]	10.6 [10.5; 10.8]	10.9 [10.8; 11.0]
Adaptativo (eBPF)	0.0 [0.0; 0.0]	6.0 [6.0; 6.0]	4.0 [3.9; 10.3]	2.7 [2.2; 4.1]

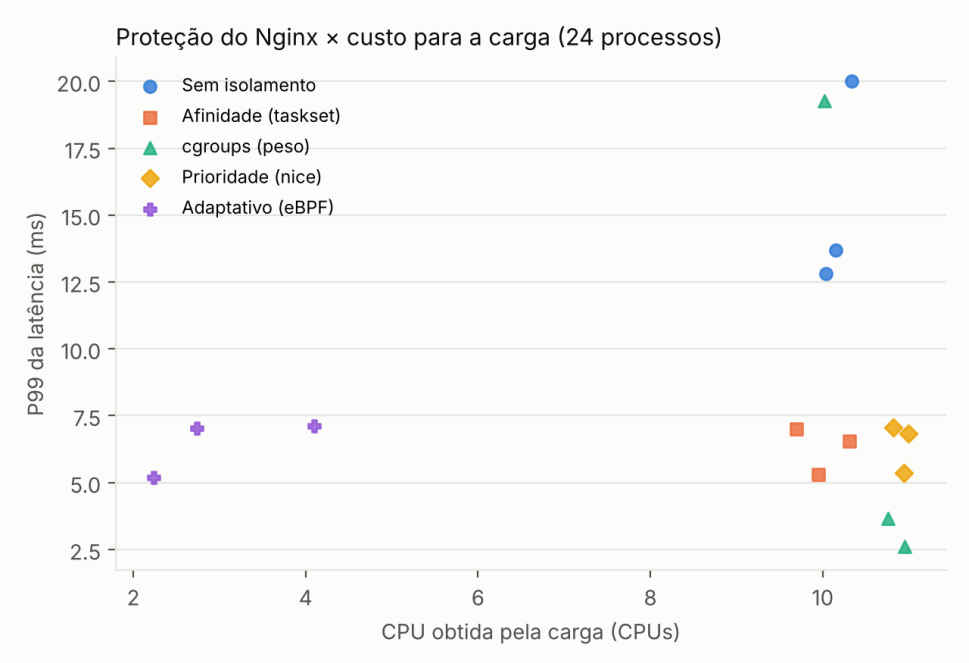


Figure 5: Proteção do Nginx (P99) contra a CPU obtida pela carga, com 24 processos.

6 Discussão

RQ1/H1. A contenção aumentou claramente a espera do Nginx (cerca de 200×), mas o P99 e o *rebuffering* não cresceram de forma monótona no piloto; H1 não foi confirmada para o P99. **RQ3/H3.** A primeira metade de H3 (a espera cresce com a contenção) foi confirmada; a segunda (acompanha o P99) não, dado $\rho \approx 0$. Hipóteses não testadas para isso: (i) o EEVDF favorece o Nginx, que dorme entre requisições, de modo que cada espera é curta (alguns milissegundos por segundo) em relação ao P99; (ii) com 12 CPUs há capacidade ociosa que absorve o servidor de uma só *thread*; (iii) o cliente Python, na mesma máquina, também disputa CPU e contribui para a latência medida; (iv) o P99 sem estresse mais alto sugere efeito de estados de baixa frequência da CPU. Por isso a coleta definitiva confina Nginx, cliente e carga a 4 CPUs e fixa o governador de CPU em *performance*, o que testa as hipóteses (ii) e (iv).

RQ2/H2. Todos os mecanismos reduziram a espera e o P99 numericamente, mas a significância depende dos dados definitivos. **RQ4/H4.** O adaptativo reduziu a espera e o P99, mas pelo corte de *cpu.max* na carga; no piloto, não custou menos que o isolamento por pesos, portanto H4 não foi sustentada. Como o primeiro degrau do controlador do piloto já aplicava um teto de 50%, o controlador foi reestruturado em duas fases (peso primeiro, teto depois), o que o aproxima do *cgroup* estático e deve reduzir o custo para a carga; isso será avaliado na coleta definitiva.

Reação. O controlador reagiu em cerca de 0,5 s após o tráfego começar, o que é adequado a execuções de 30 s mas lento para picos curtos.

7 Ameaças à Validação

- **Poucas repetições no piloto.** Com $n = 3$ nenhum teste pode ser significativo; os resultados são indícios.
- **Cliente sintético na mesma máquina.** Não reproduz um player real, e o cliente disputa CPU com o servidor e a carga.
- **Uma única máquina e kernel.** Os níveis em que o efeito aparece dependem do hardware e do escalonador (EEVDF).
- **Limiar do controlador.** Os valores padrão (0,02 e 0,005) vieram de um *container* de 2 CPUs e não foram recalibrados para a máquina de 12 CPUs.
- **Proxy de *rebuffering*.** O limite de 20 ms quase nunca foi excedido; limites menores podem ser recalculados a partir das amostras por requisição.
- **Sobrecarga do eBPF** não foi medida; o `schedstat` é o sinal do controlador.
- **Carga sintética.** O *stress-ng* é puramente de CPU e não representa uma aplicação real.
- **Variação entre rodadas.** Rodadas executadas com parâmetros diferentes (por exemplo, o intervalo entre requisições) não são comparáveis.

8 Conclusões

No piloto, a contenção de CPU aumentou cerca de 200 vezes a espera do Nginx na fila de execução, e essa espera pôde ser medida por duas vias concordantes (`schedstat` e eBPF). Os quatro mecanismos a reduziram de 5 a 15 vezes, mas a latência P99 não acompanhou a espera, e a mitigação adaptativa, apesar de reagir em cerca de 0,5 s, protegeu o servidor cortando a CPU da carga, enquanto o isolamento por pesos o fez sem esse custo. As respostas às questões de pesquisa são, portanto, preliminares: RQ1 e RQ3 parcialmente confirmadas, RQ2 numérica mas sem significância, RQ4 não sustentada.

Trabalhos seguintes: a coleta definitiva (10 repetições de 30 s) com análise estatística completa; a recalibração dos limiares; a avaliação do confinamento a 4 CPUs e do controlador em duas fases; a medição da sobrecarga do eBPF; e a avaliação de um escalonador *sched_ext*.

References

[1] Dean, J.; Barroso, L. A. The Tail at Scale. *Communications of the ACM*, v. 56, n. 2, p. 74–80, 2013.

[2] Lozi, J.-P.; Lepers, B.; Funston, J.; Gaud, F.; Quéma, V.; Fedorova, A. The Linux Scheduler: a Decade of Wasted Cores. In: *EuroSys*, 2016.

[3] Liu, L.; Wang, H.; Wang, A.; Xiao, M.; Cheng, Y.; Chen, S. Mind the Gap: Broken Promises of CPU Reservations in Containerized Multi-tenant Clouds. In: *ACM Symposium on Cloud Computing (SoCC)*, 2021.

[4] Jeffery, A.; Jensen, C.; Mortier, R. Reducing Tail Latencies Through Environment- and Neighbour-aware Thread Management. arXiv:2407.11582, 2024.

[5] Gregg, B. *BPF Performance Tools*. Addison-Wesley, 2019.

[6] Stoica, I.; Abdel-Wahab, H. Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation. Technical Report TR-95-22, Old Dominion University, 1995.

[7] Linux Kernel Documentation. Control Group v2. `https://docs.kernel.org/admin-guide/cgroup-v2.html`.

[8] Linux Kernel Documentation. PSI – Pressure Stall Information. `https://docs.kernel.org/accounting/psi.html`.

[9] Linux Kernel Documentation. Extensible Scheduler Class (sched_ext). `https://docs.kernel.org/scheduler/sched-ext.html`.

[10] Efron, B.; Tibshirani, R. J. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[11] Mann, H. B.; Whitney, D. R. On a Test of Whether One of Two Random Variables is Stochastically Larger than the Other. *Annals of Mathematical Statistics*, v. 18, n. 1, p. 50–60, 1947.

[12] Cliff, N. Dominance Statistics: Ordinal Analyses to Answer Ordinal Questions. *Psychological Bulletin*, v. 114, n. 3, p. 494–509, 1993.

[13] Holm, S. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics*, v. 6, n. 2, p. 65–70, 1979.